

“WeHeal”
an all-in-one healthcare web application

Software Requirements Specification

Prepared by

Student ID	Name
22201594	Reshad Ul Karim
22201628	Sammam Mahdi
22201574	Syeda Maliha Tabassum
22201679	Shararah Kibria

1. Introduction.....	3
1.1 Purpose.....	3
1.2 Scope.....	3
1.3 Definitions, Acronyms, and Abbreviations.....	3
1.4 References.....	3
1.5 Overview.....	4
2. Overall Description.....	4
2.1 Product Perspective.....	4
2.2 Product Features.....	4
2.3 User Classes and Characteristics.....	4
2.4 Operating Environment.....	5
2.5 Constraints.....	5
2.6 Assumptions and Dependencies.....	5
3. System Requirements.....	5
3.1 Functional Requirements.....	5
3.1.1 Authentication & Authorization.....	5
3.1.2 Patient & Doctor Interaction.....	6
3.1.3 Patient Care & Medical Services.....	6
3.1.4 Additional Services.....	6
3.2 Non-Functional Requirements.....	7
3.2.1 Performance Requirements.....	7
3.2.2 Security Requirements.....	7
3.2.3 Reliability & Availability.....	7
3.2.4 Maintainability.....	7
3.2.5 Scalability.....	7
3.3 External Interface Requirements.....	8
3.3.1 User Interfaces.....	8
3.3.2 Hardware Interfaces.....	8
3.3.3 Software Interfaces.....	8
3.3.4 Communication Interfaces.....	8
4. Technology Stack & Architectural Overview.....	8
4.1 MERN Stack Components.....	8
4.2 High-Level Architecture.....	8
5. Tentative Development Plan (Agile Methodology).....	9
Sprint 1 (Weeks 1–2): Core Setup & Authentication.....	9
Sprint 2 (Weeks 3–4): Appointment & Telemedicine System.....	9
Sprint 3 (Weeks 5–6): Patient Care & Emergency Services.....	10
Sprint 4 (Week 7): Payment & Additional Services.....	10
Sprint 5 (Weeks 8–9): Service Quality & Final Touches.....	11
6. Acceptance Criteria.....	11
7. Conclusion.....	11

1. Introduction

1.1 Purpose

This Software Requirement Specification (SRS) document outlines the requirements for developing “**WeHeal**” - **an all-in-one healthcare web application** using the MERN stack (MongoDB, Express.js, React.js, Node.js). The primary goal of this application is to facilitate a secure and efficient healthcare ecosystem, enabling patients to find nearby clinics/hospitals, schedule appointments, receive telemedicine services, manage medical records, and process payments—all while ensuring compliance with relevant healthcare regulations .

1.2 Scope

The scope of this project includes the design, development, testing, and deployment of the healthcare web application, catering to:

- **Patients** who can register, book appointments, access electronic medical records (EMR), and receive telemedicine services.
- **Doctors** who can manage appointments, conduct telemedicine sessions, maintain patient records, and handle prescriptions.
- **Clinic Staff** who can manage appointments, generate reports, and assist with administrative tasks.
- **Administrators** who oversee system operations, manage user roles, and maintain compliance standards.

This SRS covers the functional and non-functional requirements for the project, the constraints, assumptions, high-level design, and the development plan following Agile methodology.

1.3 Definitions, Acronyms, and Abbreviations

- **MERN**: MongoDB, Express.js, React.js, Node.js.
- **EMR**: Electronic Medical Records.
- **HIPAA**: Health Insurance Portability and Accountability Act (U.S. standard for healthcare data privacy).
- **GDPR**: General Data Protection Regulation (European Union data privacy and security law).
- **OTP**: One-Time Password.
- **RBAC**: Role-Based Access Control.

1.4 References

- MERN Stack documentation:
 - [MongoDB](#)
 - [Express.js](#)
 - [React.js](#)
 - [Node.js](#)

1.5 Overview

Section 2 provides an overall description of the product, including its user classes and environment. Section 3 details the functional and non-functional requirements. Section 4 outlines the technology stack and architectural overview. Section 5 presents the development plan, including sprint-wise breakdown and deliverables.

2. Overall Description

2.1 Product Perspective

The healthcare web application is a standalone system built on the MERN stack. It integrates with external APIs or services where necessary (e.g., payment gateways, SMS/email gateways, telemedicine integrations). The application must handle high levels of security and maintain data privacy in accordance with HIPAA/GDPR.

2.2 Product Features

1. **Location Services:** Real-time display of nearby clinics, hospitals, and doctors based on a patient's location.
2. **OTP-Based Authentication:** Secure login and registration for both patients and doctors.
3. **Role Management:** Differentiated dashboards for patients, doctors, staff, and admin users.
4. **Appointment & Telemedicine:** Scheduling, rescheduling, and cancellation of appointments; real-time doctor availability; video consultations via WebRTC or a third-party provider (e.g., Twilio).
5. **Notifications:** Automated SMS/email/push reminders for upcoming appointments, prescription pickups, and follow-up.
6. **EMR & Prescription Management:** Securely store medical records; generate and share e-prescriptions.
7. **Emergency Services:** Quick ambulance booking, emergency contact lists, and nurse support for urgent care.
8. **Payment & Insurance Integration:** Support for online payments, insurance claims, and reimbursements.
9. **Medicine & Lab Services:** Online medicine ordering and lab test bookings, with delivery options.
10. **Feedback & Ratings:** Patients can rate doctors, share feedback, and view ratings to facilitate service quality.

2.3 User Classes and Characteristics

1. **Patients:** Individuals seeking medical consultations, lab tests, emergency services, etc. Typically non-technical, requiring a user-friendly interface and secure data handling.

2. **Doctors:** Healthcare providers who manage appointments, conduct telemedicine, update EMRs, and generate prescriptions. Require an intuitive interface to manage multiple patients efficiently.
3. **Clinic Staff:** Administrative staff that handles scheduling, reception, and patient coordination. They need access to appointment management features and limited patient data.
4. **Administrators:** System overseers responsible for user management, compliance checks, and application maintenance. Have full access to system settings and analytics dashboards.

2.4 Operating Environment

- **Web Application:** Accessible on modern web browsers (Chrome, Firefox, Edge, Safari).
- **Mobile Compatibility:** Responsive design for smartphones and tablets.
- **Server-Side:** Node.js and Express.js running in a containerized or cloud environment.
- **Database:** MongoDB cluster (preferably cloud-managed for scalability and backups).
- **Hosting:** Deployment could be on AWS, Azure, Google Cloud, or any equivalent production environment.

2.5 Constraints

- **Regulatory Compliance:** Must adhere to HIPAA/GDPR for handling patient data.
- **Security:** Must implement encryption at rest and in transit (HTTPS/SSL, database encryption).
- **Performance:** The system must support a large user base with minimal latency for telemedicine and real-time updates.
- **Scalability:** The system architecture should allow horizontal scaling to handle peak loads.
- **Time & Resource:** The project is to be delivered in approximately 8–9 weeks with a team of four developers.

2.6 Assumptions and Dependencies

- Users have a stable internet connection for telemedicine and streaming.
- Third-party services for payment, SMS/email, and video conferencing are available and reliable.
- Mobile users will grant access to GPS for location-based services when prompted.
- Clinic staff are knowledgeable in using web-based tools and can be trained if needed.

3. System Requirements

3.1 Functional Requirements

3.1.1 Authentication & Authorization

1. **OTP Authentication:**
 - **FR-1:** The system shall generate and send a time-bound OTP to the user's email or phone number for registration and login.
 - **FR-2:** The system shall invalidate OTPs after a configurable time period (e.g., 5 minutes).

2. Role-Based Access Control (RBAC):

- **FR-3:** The system shall provide different dashboards for patients, doctors, clinic staff, and admins.
- **FR-4:** The system shall restrict each user's access to features according to their role (e.g., staff cannot access full patient EMR).

3.1.2 Patient & Doctor Interaction

1. Appointment Management:

- **FR-5:** Patients shall be able to search for doctors by specialty, location, or availability.
- **FR-6:** Patients shall be able to schedule, reschedule, or cancel appointments.
- **FR-7:** Doctors shall be able to update their availability in real-time (e.g., "available," "busy," "offline").
- **FR-8:** The system shall display a virtual queue for walk-in patients with real-time status updates.

2. Telemedicine Integration:

- **FR-9:** The system shall support secure voice/video calls for remote consultations.
- **FR-10:** The system shall store basic call logs and conversation summaries without violating privacy regulations.
- **FR-11:** The system shall enable secure in-app messaging for doctor-patient communication.

3.1.3 Patient Care & Medical Services

1. Electronic Medical Records (EMR):

- **FR-12:** The system shall maintain a secure, encrypted record of all patient medical data, including prior consultations.
- **FR-13:** Doctors and authorized staff shall be able to view and update EMR details (e.g., diagnosis, treatment plan).

2. Prescription Management:

- **FR-14:** The system shall allow doctors to generate e-prescriptions and share them with patients or partner pharmacies.
- **FR-15:** Patients shall be able to view their prescription history in their dashboard.

3. Emergency Services:

- **FR-16:** Patients shall have a quick-access feature to emergency contacts or facilities.
- **FR-17:** The system shall allow ambulance booking and real-time tracking if integrated with external providers.
- **FR-18:** Nurses or specialized staff shall have an interface for emergency service requests (oxygen, wheelchair).

3.1.4 Additional Services

1. Payments & Insurance Integration:

- **FR-19:** The system shall integrate with at least one major payment gateway (Stripe, PayPal, etc.).

- **FR-20:** Users shall have the option to link health insurance for streamlined claims processing.
- 2. **Medicine & Lab Services:**
 - **FR-21:** Patients shall be able to order prescribed medicines and track delivery.
 - **FR-22:** Patients shall be able to book lab tests and view results through the app.
- 3. **Reward System:**
 - **FR-23:** The system shall allow loyalty points, discounts, or referral bonuses for regular or referred users.
- 4. **Feedback & Ratings:**
 - **FR-24:** Patients shall be able to provide feedback and ratings for doctors and services.
 - **FR-25:** Admin shall have access to an aggregated view of feedback for quality monitoring.

3.2 Non-Functional Requirements

3.2.1 Performance Requirements

- **NFR-1:** The system should handle multiple concurrent telemedicine sessions without significant degradation in performance.
- **NFR-2:** Average page load time shall not exceed a defined time limit under normal network conditions.

3.2.2 Security Requirements

- **NFR-3:** All communication between client and server shall occur over HTTPS (SSL/TLS).
- **NFR-4:** Sensitive fields in the database (e.g., patient records, payment details) shall be encrypted at rest.
- **NFR-5:** The system must comply with regulations regarding data privacy, consent, and breach notifications.

3.2.3 Reliability & Availability

- **NFR-6:** The system shall maintain 99.9% uptime, excluding scheduled maintenance.
- **NFR-7:** Data backups shall be taken daily to ensure minimal data loss in case of catastrophic failure.

3.2.4 Maintainability

- **NFR-8:** The codebase shall follow consistent naming conventions and standard design patterns to ease future maintenance.
- **NFR-9:** Documentation of APIs, data models, and architecture shall be kept current.

3.2.5 Scalability

- **NFR-10:** The system architecture shall accommodate horizontal scaling of the Node.js application and MongoDB cluster with minimal reconfiguration.

3.3 External Interface Requirements

3.3.1 User Interfaces

- **UI-1:** A responsive web interface (React.js) that adjusts to desktop, tablet, and mobile viewports.
- **UI-2:** Intuitive navigation with separate dashboards for each user role.

3.3.2 Hardware Interfaces

- **HI-1:** None specific; the system will run on common server hardware or cloud-based instances.

3.3.3 Software Interfaces

- **SI-1:** Payment Gateway APIs (Stripe, PayPal, or equivalent).
- **SI-2:** SMS/email services for OTP and notifications (Twilio, SendGrid, or equivalent).
- **SI-3:** Video conferencing services (WebRTC-based or third-party SDK).

3.3.4 Communication Interfaces

- **CI-1:** RESTful APIs will be used for communication between front-end (React.js) and back-end (Node.js/Express.js).

4. Technology Stack & Architectural Overview

4.1 MERN Stack Components

1. **MongoDB:** Document-oriented database for storing user profiles, appointment data, EMRs, etc.
2. **Express.js:** Node.js framework for building the RESTful API and handling server-side logic.
3. **React.js:** Front-end library for building a responsive and interactive user interface.
4. **Node.js:** JavaScript runtime for executing the back-end application.

4.2 High-Level Architecture

1. **Presentation Layer (React.js):** Handles user interaction, form submissions, and displays data fetched from the API.
2. **Business Logic Layer (Express.js/Node.js):** Processes requests, enforces business rules, and orchestrates data flow.
3. **Data Layer (MongoDB):** Stores all persistent data, including user accounts, appointments, EMRs, etc.

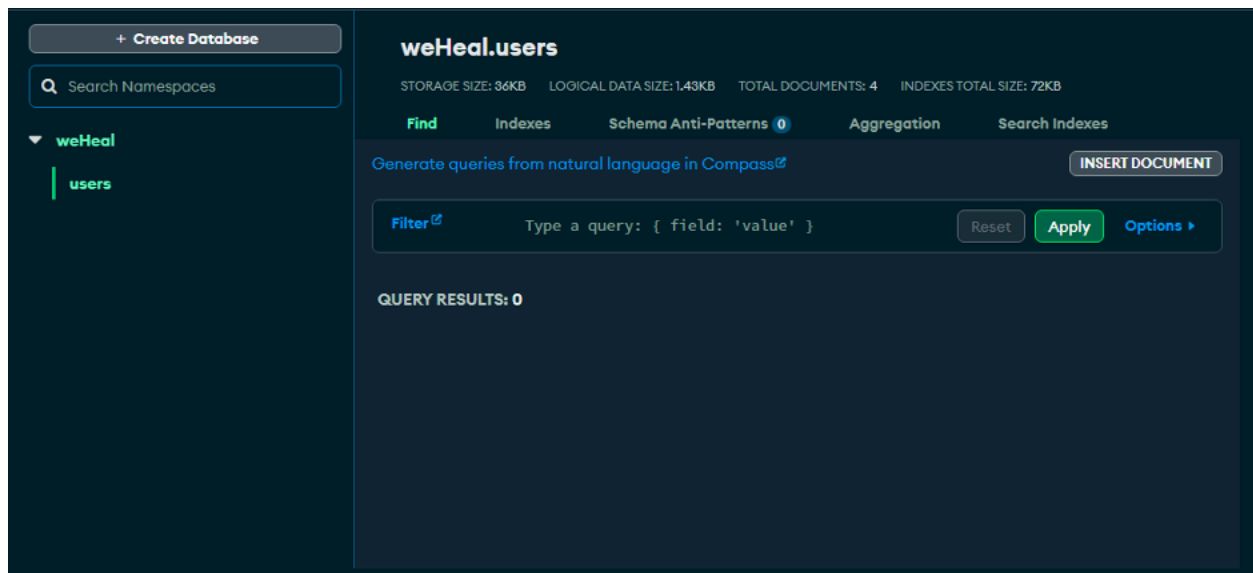
6. Conclusion

This SRS provides a detailed outline for the design and development of “WeHeal”—a comprehensive healthcare web application using the MERN stack. It ensures a patient-centric approach by integrating core healthcare functionalities—scheduling, telemedicine, EMR, and emergency services—while maintaining strict data privacy and security compliance. By following the outlined Agile sprints, the project aims to deliver a robust, scalable, and user-friendly solution within the designated timeline.

SPRINT 2

Feature 1 & 2: Registration and OTP Authentication

- Primarily the user table is empty before registering a user



- The user registers

Sign Up

Samman

sammanpersonal@gmail.com

....

01822926571

Doctor

Sign Up

Already have an account?

Login

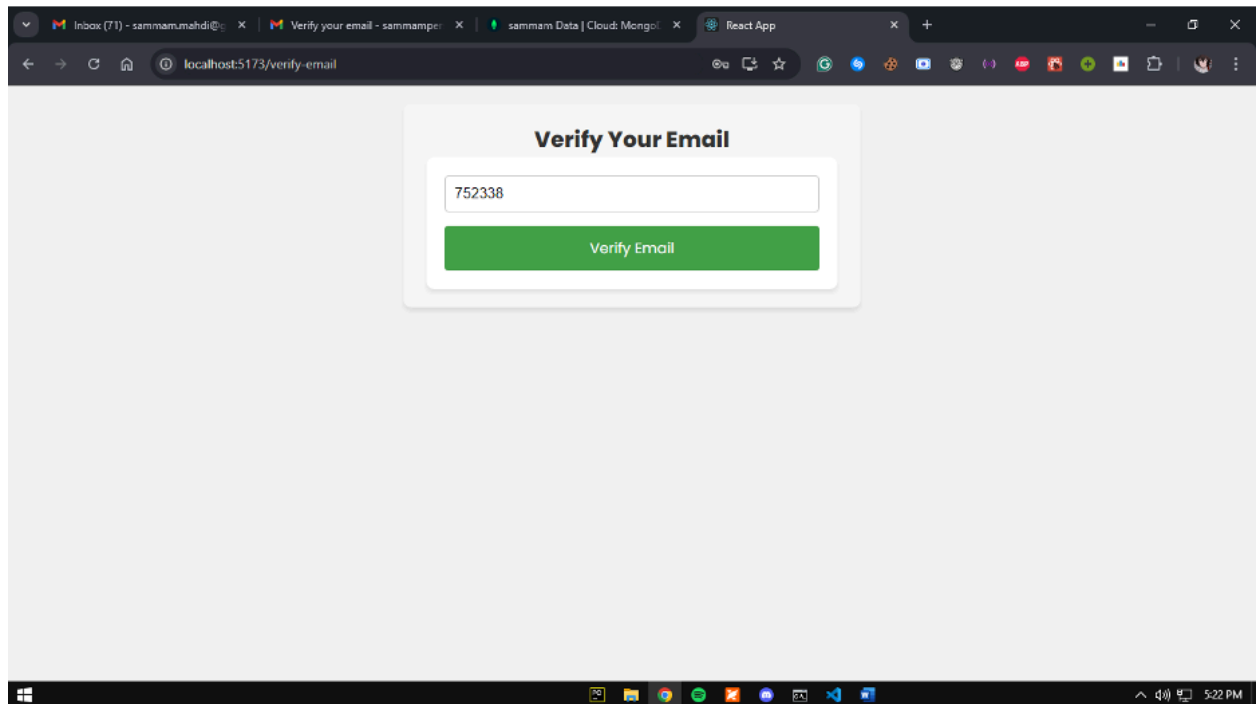
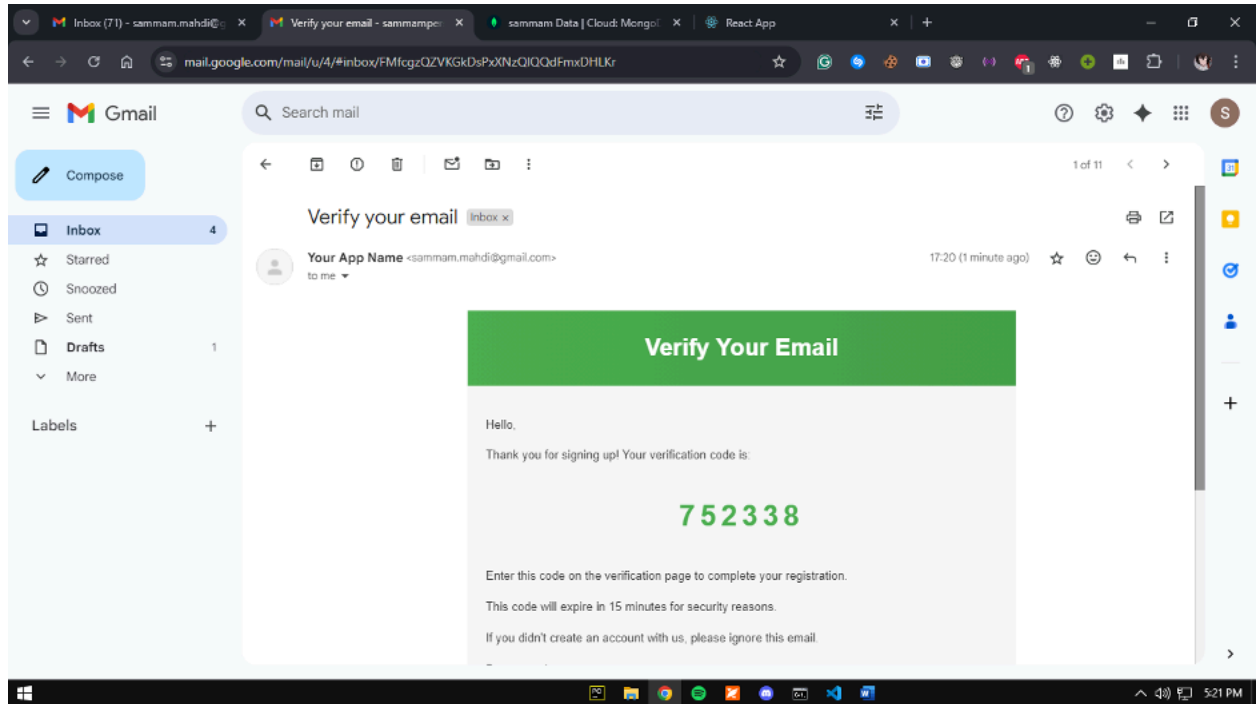
- After the user has given their information, they are redirected to the email verification page (An OTP is sent to the user email). The OTP expires in 1 hour.

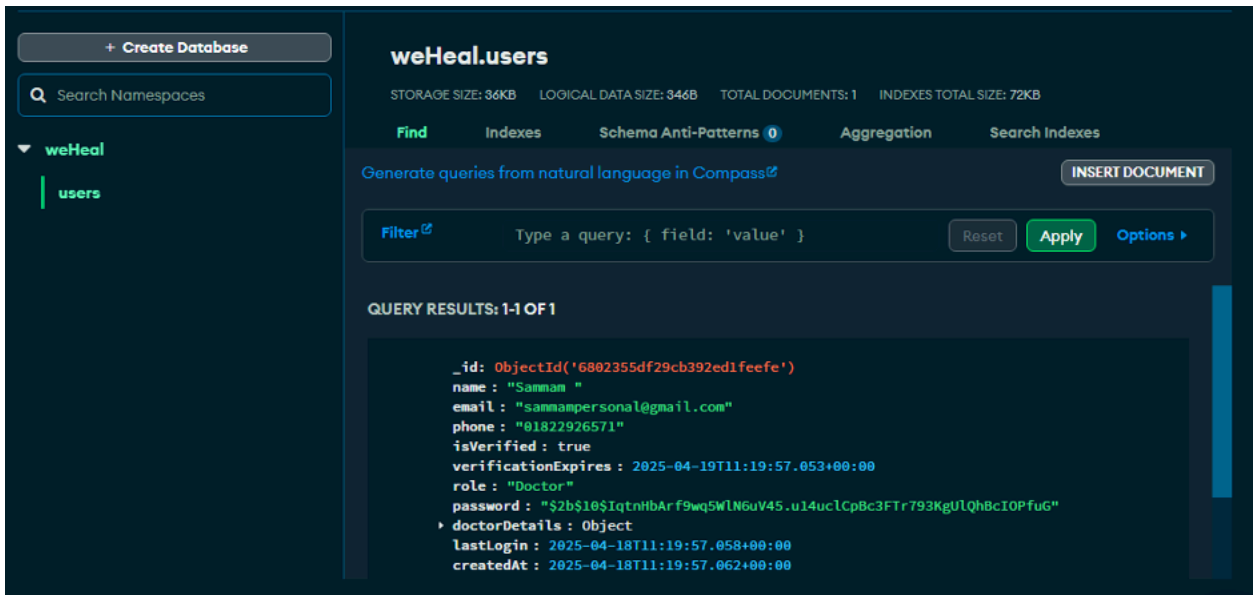
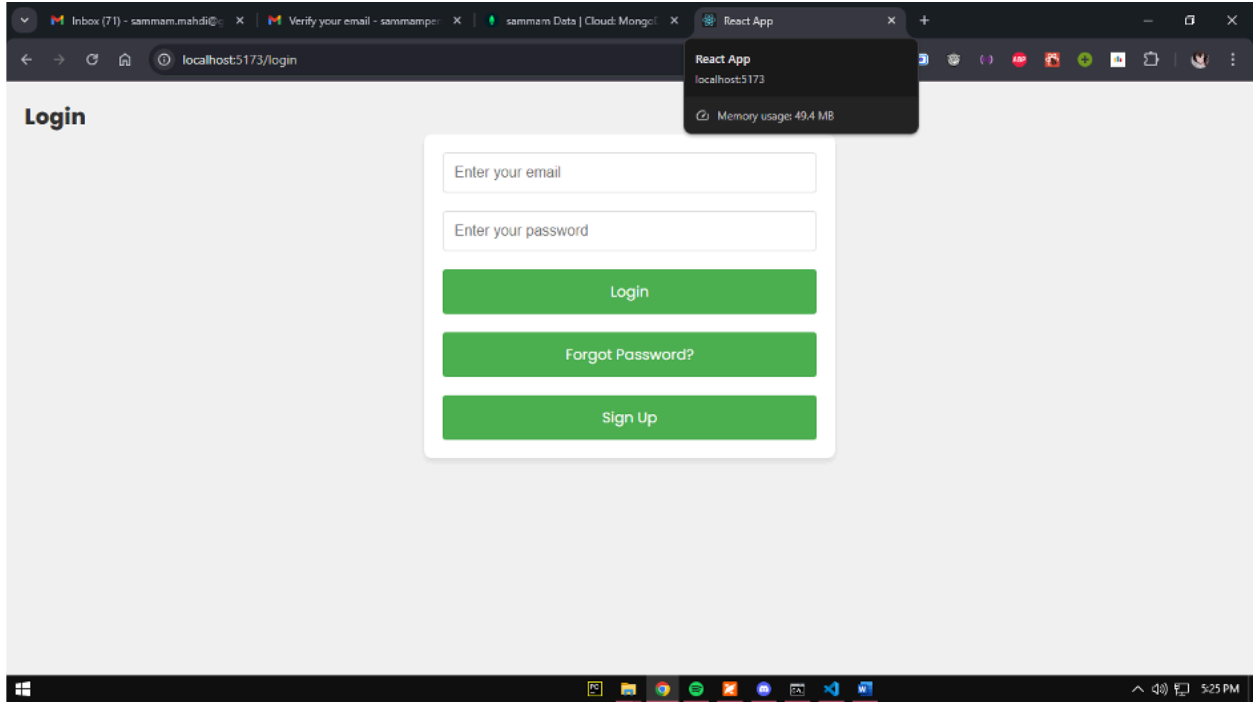
Verify Your Email

Enter verification code

Verify Email

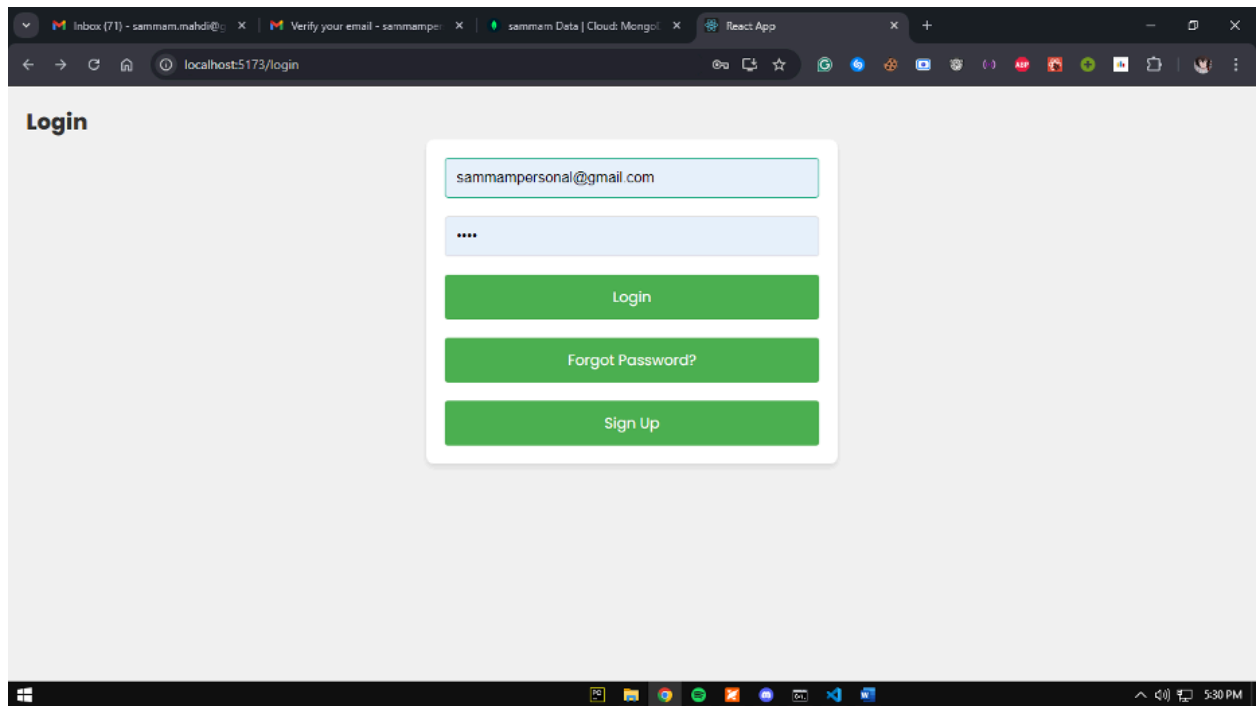
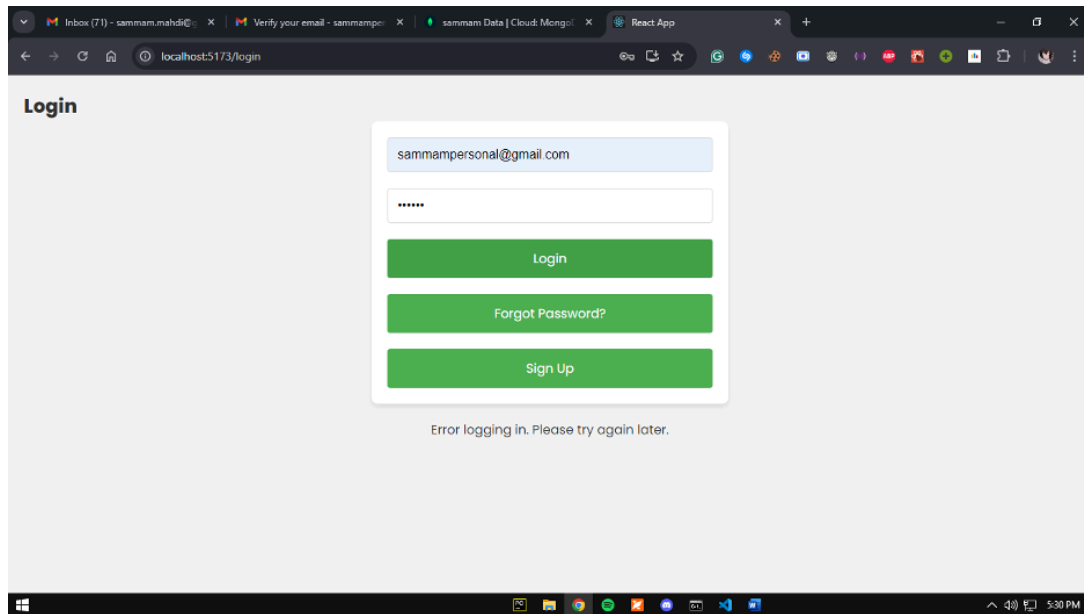
- After entering the correct OTP, the user is redirected to the login page and the user information is successfully populated in the database.





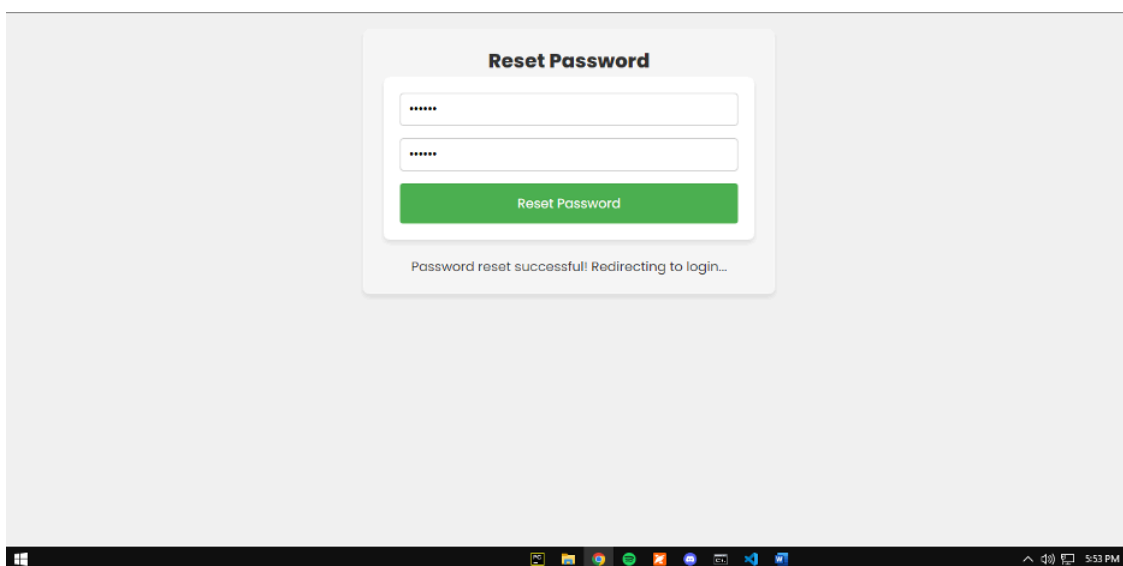
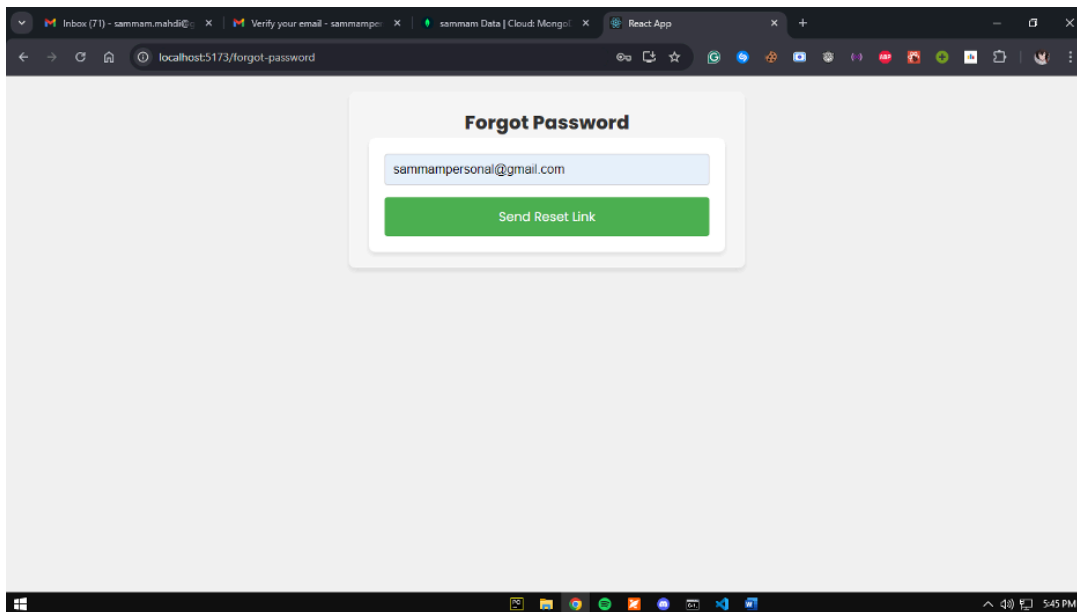
Feature 3: Login

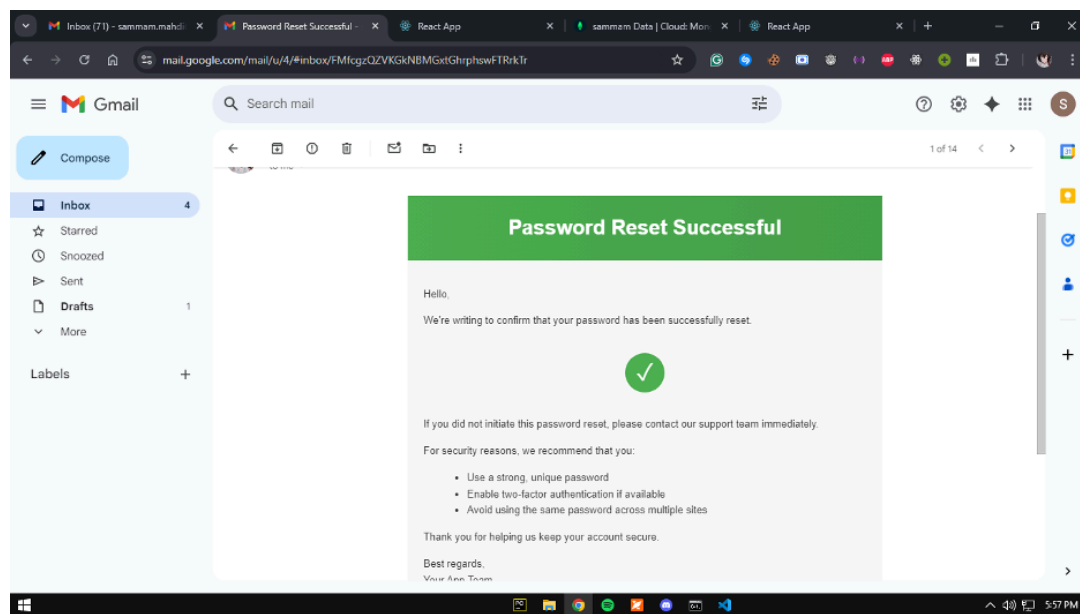
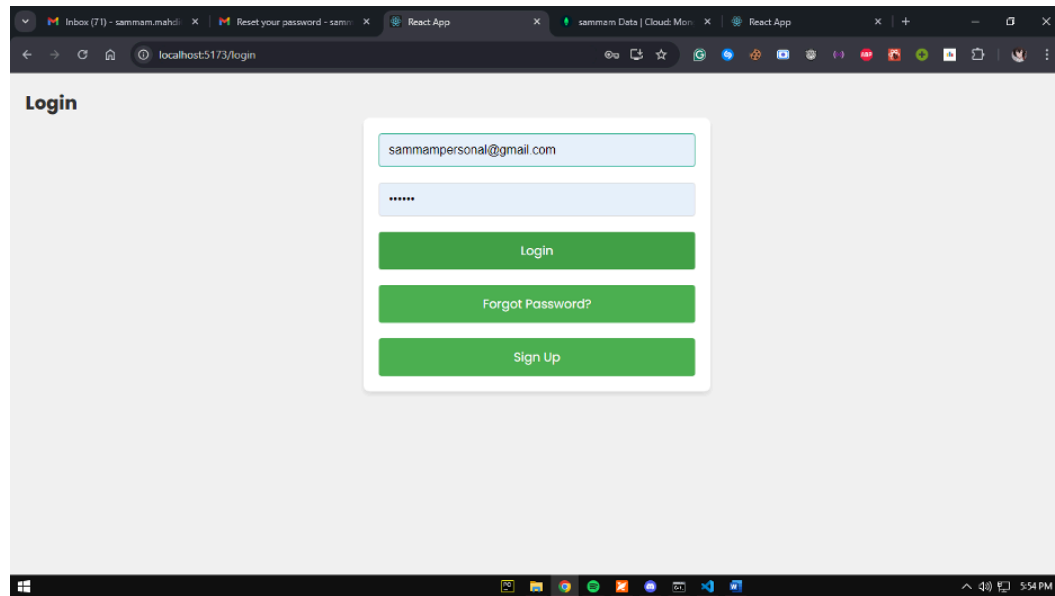
- The user will successfully able to login to the website if they provide the correct credentials else, they will be denied access to the website.



Feature 4: Forgot Password and Password Reset

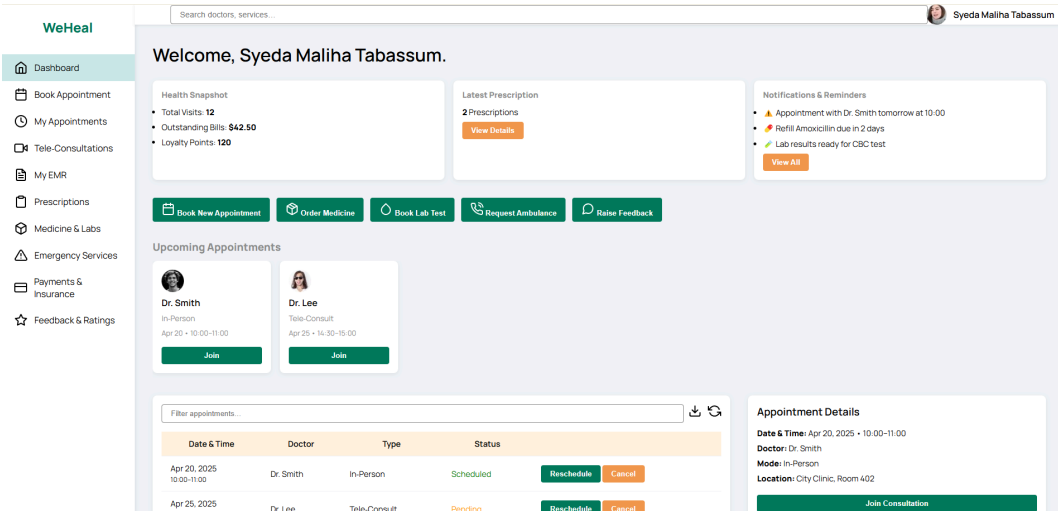
If the user forgets their password, then they can click on the forget password button and go to the forgot password page and if they have provided their email correctly then a password reset email will be sent to their email that has a password reset link with a specific token. If only the token is correct only then the user may successfully reset their password the token also has an expiration deadline of 1 hour.





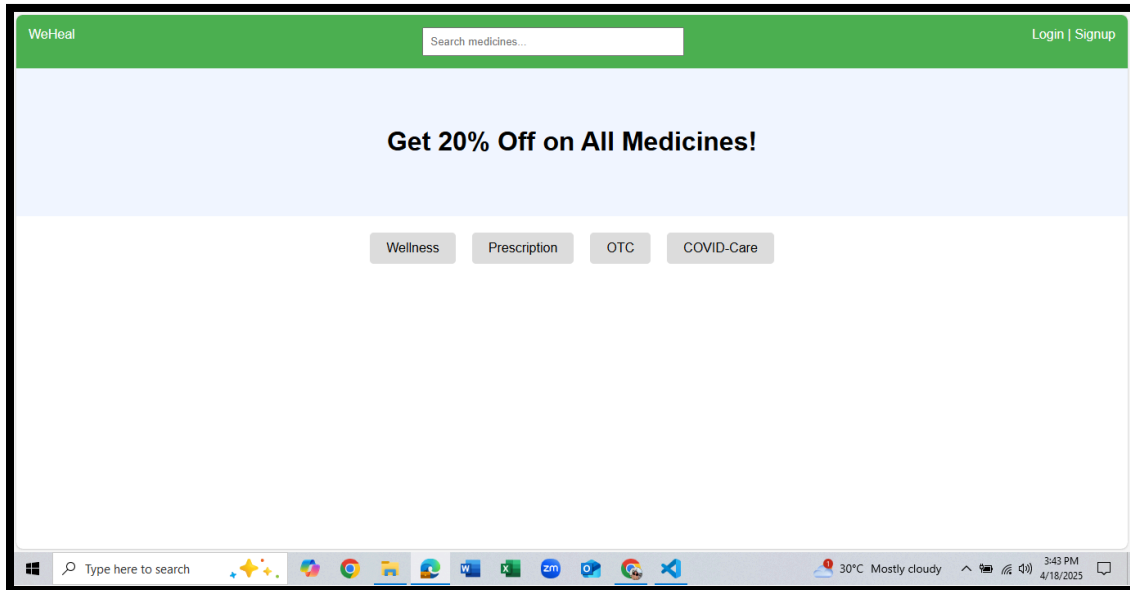
Feature 5: User-end dashboard

a) Patient Dashboard

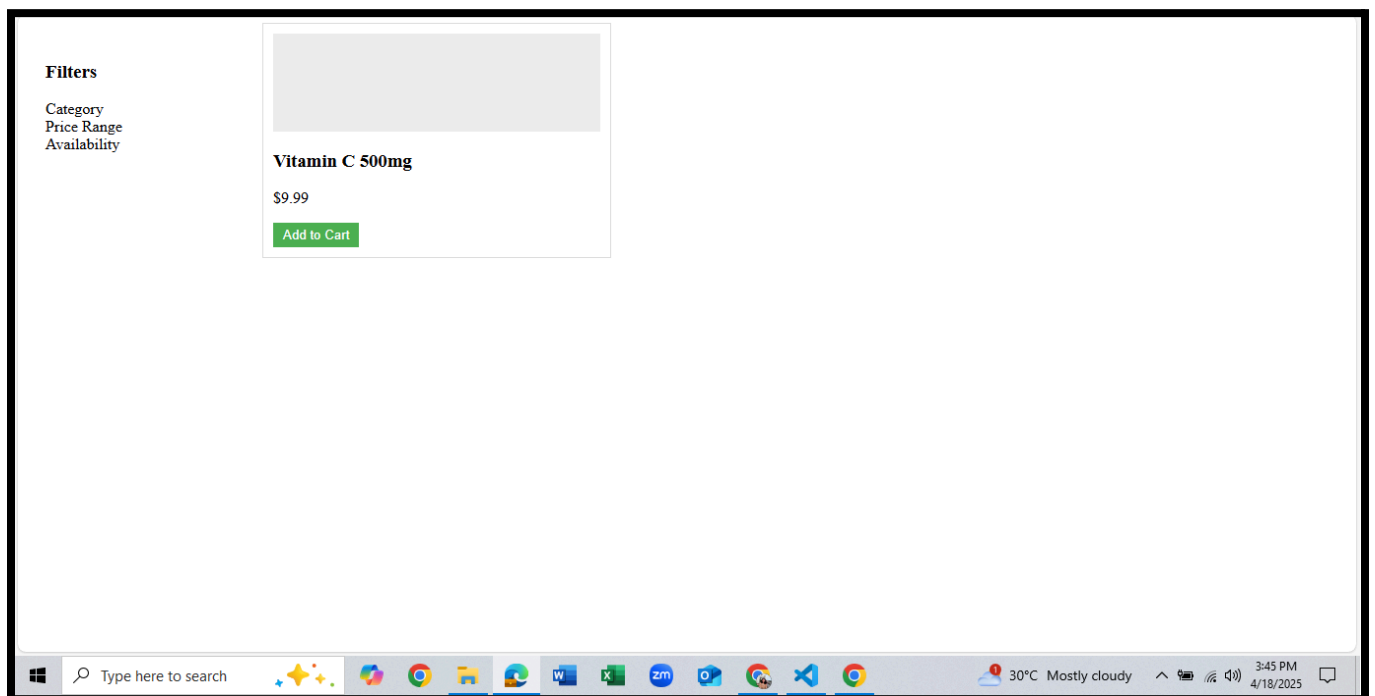


SPRINT 3

Feature 6: Marketplace dashboard



Feature 7: Product list



Feature 8: Cart

Product	Price	Quantity
Vitamin C 500mg	\$9.99	1
Total	\$34.97	

[Proceed to Checkout](#)

Feature 9: Delivery placement

Delivery Address

Payment Method

☐ Credit Card ☐ PayPal

Feature: 10 Admin Dashboard

